

# LAPORAN QUIZ 2

## PEMROGRAMAN PERANGKAT BERGERAK

*Aplikasi Pemesanan Makanan & Minuman Kantin "Cathering"*  
(React Native – Expo)

**Nama** : Steve Valentino Wijaya  
**NPM** : 242310026  
**Kelas** : TI-24-KA  
**Mata Kuliah** : Pemrograman Perangkat Bergerak

### 1. Pendahuluan

Laporan ini merupakan dokumentasi hasil pengerjaan Quiz 2 mata kuliah Pemrograman Perangkat Bergerak, berupa aplikasi mobile bernama "Cathering" yang dibangun menggunakan React Native (Expo). Nama "Cathering" merupakan plesetan dari kata Catering dan Cathering (canteen + gathering), yaitu aplikasi pemesanan makanan dan minuman kantin secara digital.

Aplikasi dijalankan menggunakan Expo Go versi 57.0.2 dengan Expo SDK 57, React Native 0.86, dan React 19.2. Berikut adalah beberapa library utama yang digunakan:

- @react-navigation/native & bottom-tabs → navigasi antar halaman (bottom tab)
- @react-native-async-storage/async-storage → menyimpan sesi login & riwayat transaksi secara lokal
- expo-linear-gradient → efek gradient pada header halaman
- @expo/vector-icons (Ionicons) → ikon pada tab bar & tombol aksi
- FakeStoreAPI (<https://fakestoreapi.com>) → autentikasi login (POST /auth/login)

Alur aplikasi secara umum: pengguna login menggunakan akun dari FakeStoreAPI, kemudian dapat melihat serta mencari menu kantin, menambahkannya ke keranjang, melakukan checkout, melihat riwayat transaksi, dan melihat ringkasan aktivitas pada halaman profil.

### 2. Struktur Folder Proyek

```
cathering/
├── App.js           # Entry point, providers & navigasi
├── theme.js        # Warna, shadow, radius global
├── data/
│   └── menuData.js # Data menu makanan & minuman
├── context/
│   ├── AuthContext.js # Login/logout via FakeStoreAPI
│   ├── CartContext.js # State keranjang & perhitungan total
│   └── OrderContext.js # Riwayat pesanan/transaksi
├── components/
│   ├── MenuCard.js    # Kartu item menu
│   └── CategoryPill.js # Tombol filter kategori
└── screens/
    ├── LoginScreen.js
    ├── MenuScreen.js
    ├── CartScreen.js
    ├── HistoryScreen.js
    └── ProfileScreen.js
```

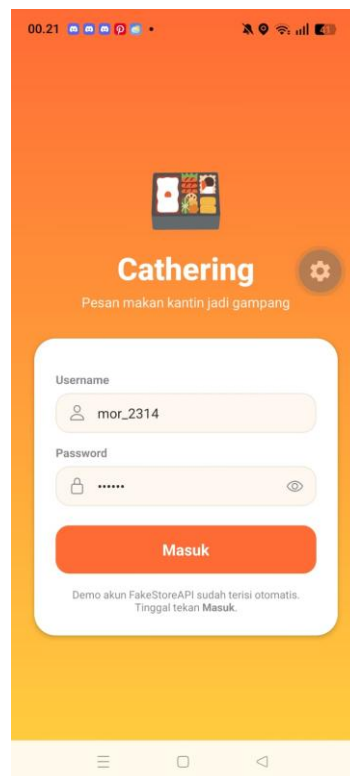
### 3. Autentikasi (Login)

Halaman Login (screens/LoginScreen.js) menyediakan form username dan password yang sudah otomatis terisi dengan akun demo resmi FakeStoreAPI (mor\_2314 / 83r5^\_), sehingga pengguna cukup menekan

tombol “Masuk”. Proses autentikasi ditangani oleh AuthContext (context/AuthContext.js) yang memanggil endpoint POST <https://fakestoreapi.com/auth/login>.

```
const login = async (username, password) => {
  const res = await fetch("https://fakestoreapi.com/auth/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });
  if (!res.ok) throw new Error("Username atau password salah");
  const data = await res.json();
  setToken(data.token);
  setUser(userProfile);
  await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify({ user: userProfile, token: data.token }));
};
```

Jika login berhasil, token dan data user disimpan secara permanen menggunakan AsyncStorage (key @cathering\_auth), sehingga saat aplikasi dibuka kembali pengguna tidak perlu login ulang selama sesi belum dihapus (logout).

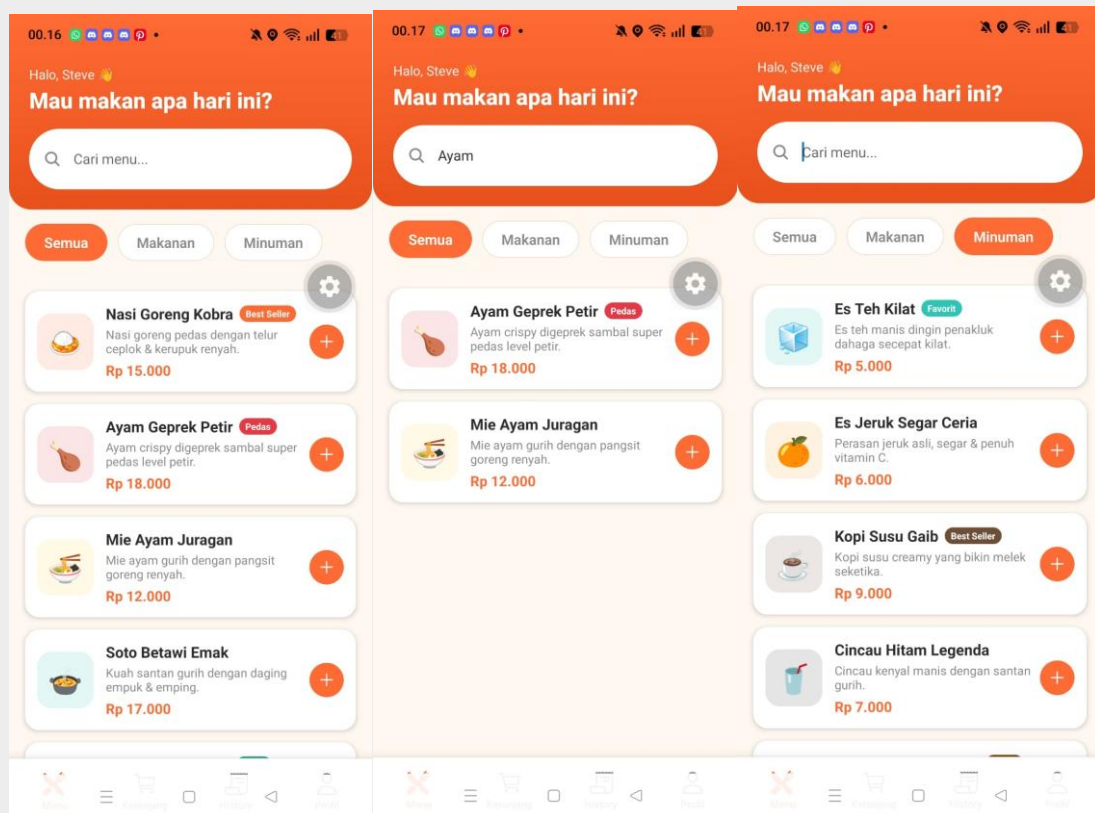


## 4. Halaman Menu (List Menu & Pencarian)

Halaman Menu (screens/MenuScreen.js) menampilkan daftar menu makanan dan minuman kantin dalam bentuk kartu (components/MenuCard.js), lengkap dengan emoji, harga, badge (Best Seller/Pedas/Sehat/dll), tombol tambah/kurang jumlah, kolom pencarian, dan filter kategori (Semua/Makanan/Minuman) menggunakan komponen CategoryPill.

```
const filtered = useMemo(() => {
  return MENU_DATA.filter((m) => {
    const matchCategory = category === "Semua" || m.category === category;
    const matchSearch = m.name.toLowerCase().includes(search.toLowerCase());
    return matchCategory && matchSearch;
  });
}, [category, search]);
```

Data menu disimpan secara statis pada `data/menuData.js`, berisi nama menu, kategori, harga, emoji, deskripsi, badge, dan warna tema untuk setiap item. Saat pengguna menambahkan item ke keranjang, muncul cart bar mengambang (floating) di bagian bawah layar yang menampilkan jumlah item dan total harga, serta tombol untuk langsung menuju halaman Keranjang.



Halaman Menu, pencarian & filter kategori

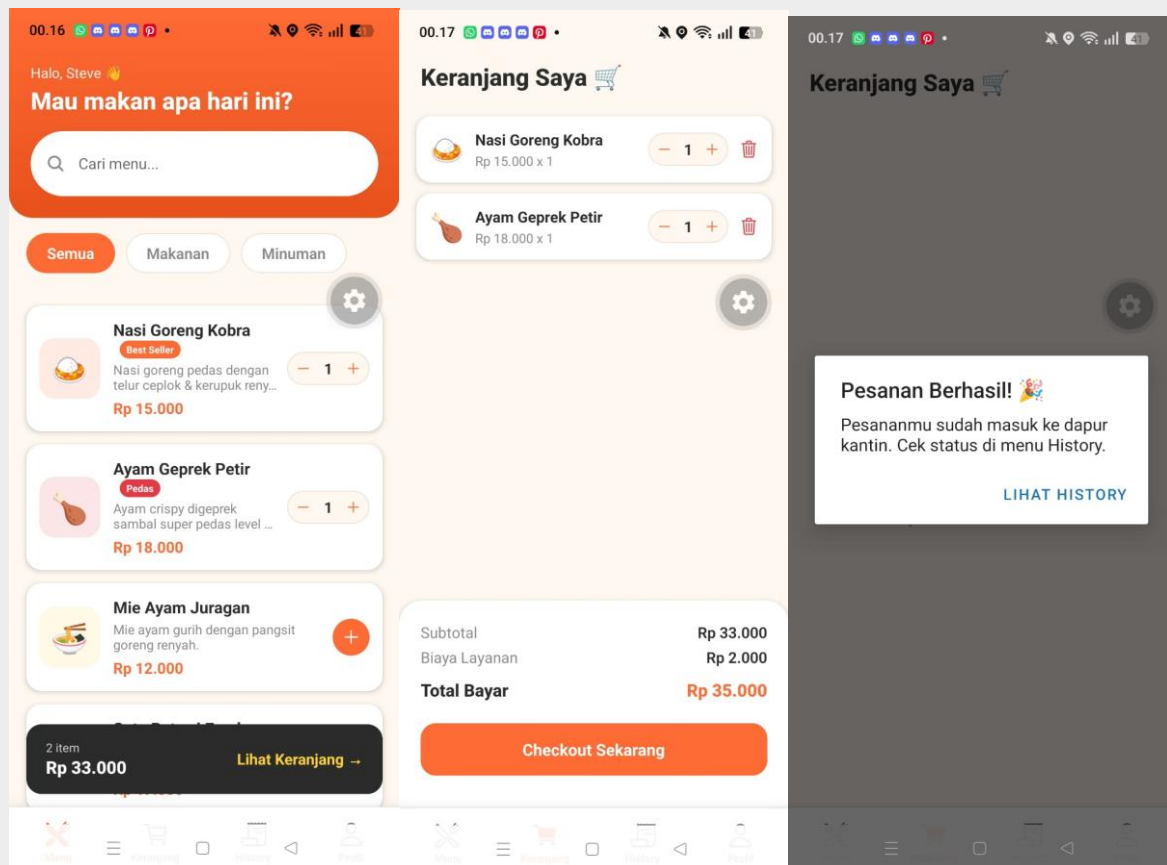
## 5. Keranjang & Checkout

State keranjang dikelola oleh `CartContext` (`context/CartContext.js`) yang menyediakan fungsi `addItem`, `decreaseItem`, `removeItem`, `clearCart`, serta perhitungan otomatis `totalPrice` dan `totalItems` menggunakan `useMemo`.

```
const totalPrice = useMemo(
  () => items.reduce((sum, i) => sum + i.price * i.qty, 0),
  [items]
);
```

Pada halaman Keranjang (`screens/CartScreen.js`), pengguna dapat mengubah jumlah tiap item, melihat ringkasan Subtotal, Biaya Layanan (Rp 2.000), dan Total Bayar. Saat menekan tombol “Checkout Sekarang”, item pada keranjang disimpan sebagai transaksi baru melalui `OrderContext`, keranjang dikosongkan, dan muncul notifikasi keberhasilan pesanan.

```
const handleCheckout = async () => {
  await addOrder(items, grandTotal);
  clearCart();
  Alert.alert("Pesanan Berhasil! \uD83C\uDF89", "...", [
    { text: "Lihat History", onPress: () => navigation.navigate("History") },
  ]);
};
```



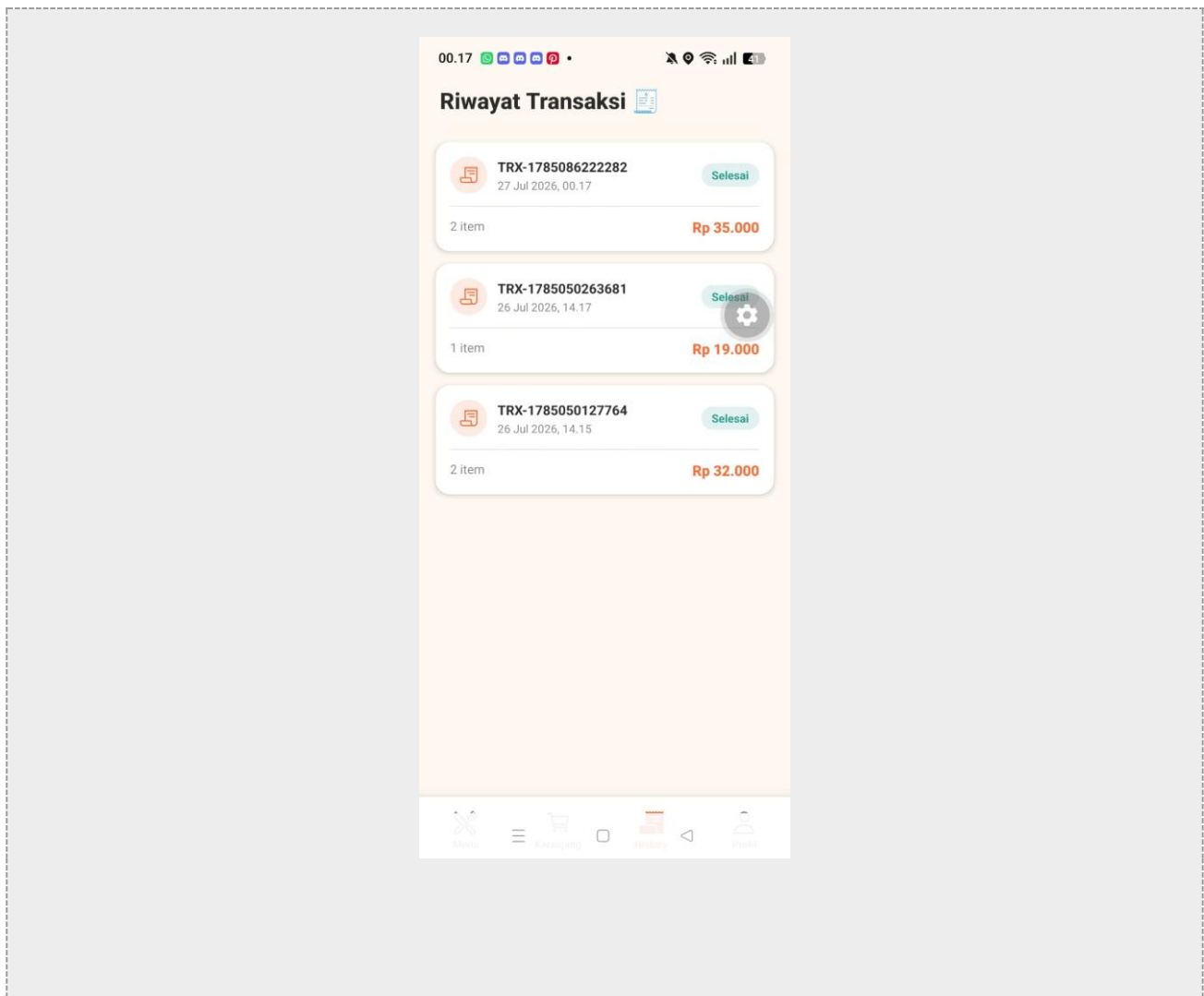
Halaman Keranjang & ringkasan checkout

## 6. Riwayat Transaksi (History)

OrderContext (context/OrderContext.js) menyimpan setiap transaksi yang sudah checkout ke AsyncStorage (key @cathering\_orders) secara permanen, lengkap dengan ID transaksi (format TRX-<timestamp>), tanggal, daftar item, total bayar, dan status.

```
const addOrder = async (cartItems, total) => {
  const newOrder = {
    id: `TRX-${Date.now()}`,
    date: new Date().toISOString(),
    items: cartItems,
    total,
    status: "Selesai",
  };
  const updated = [newOrder, ...orders];
  setOrders(updated);
  await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
};
```

Halaman History (screens/HistoryScreen.js) menampilkan daftar transaksi dalam bentuk kartu yang dapat ditekan (expand/collapse) untuk melihat rincian item beserta subtotal masing-masing item pada transaksi tersebut.



*Halaman History*

## 7. Profil Pengguna

Halaman Profil (screens/ProfileScreen.js) menampilkan data pengguna hasil login (nama & email), serta ringkasan aktivitas berupa Total Pesanan dan Total Belanja yang dihitung dari data pada OrderContext. Tersedia juga tombol Keluar yang akan menampilkan dialog konfirmasi sebelum menghapus sesi login (logout).

```
const totalSpent = orders.reduce((s, o) => s + o.total, 0);

const confirmLogout = () => {
  Alert.alert("Keluar akun?", "Kamu perlu login lagi untuk memesan.", [
    { text: "Batal", style: "cancel" },
    { text: "Keluar", style: "destructive", onPress: logout },
  ]);
};
```



Halaman Profil